

Automatic Differentiation

Benoît Legat

☐ Full Width Mode ☐ Present Mode

☰ **Table of Contents**

Differentiation approaches

Chain rule

Forward Differentiation

Reverse Differentiation

Comparison

Discontinuity

Neural network

Wine example 🍷

Second-order

Differentiation approaches

We can compute partial derivatives in different ways:

1. **Symbolically**, by fixing one of the variables and differentiating with respect to the others, either manually or using a computer.
2. **Numerically**, using the formula $f'(x) \approx (f(x + h) - f(x))/h$.
3. **Algorithmically**, either forward or reverse : this is what we will explore here.

Chain rule ⇔

Consider $f(x) = f_3(f_2(f_1(x)))$. If we don't have the expression of f_1 but we can only evaluate $f_i(x)$ or $f'(x)$ for a given x ? The chain rule gives

$$f'(x) = f'_3(f_2(f_1(x))) \cdot f'_2(f_1(x)) \cdot f'_1(x).$$

Let's define $s_0 = x$ and $s_k = f_k(s_{k-1})$, we now have:

$$f'(x) = f'_3(s_2) \cdot f'_2(s_1) \cdot f'_1(s_0).$$

Two choices here:

Forward	Reverse
$t_0 = 1$	$r_3 = 1$
$t_k = f'_k(s_{k-1}) \cdot t_{k-1}$	$r_k = r_{k+1} \cdot f'_{k+1}(s_k)$

Forward Differentiation ↗

Error message

InterruptedException:

```
1 img("Blondel_Rouvet_Figure_8_1", :height => 250)
```

👁 Reading hidden code

Figure 8.1

Implementation ↗

```
1 struct Dual{T}
2     value::T # s_k
3     derivative::T # t_k
4 end
```

```
1 Base.:- (x::Dual{T}) where {T} = Dual(-x.value, -x.derivative)
```

```
1 Base.:*(x::Dual{T}, y::Dual{T}) where {T} = Dual(x.value * y.value, x.value *
y.derivative + x.derivative * y.value)
```

► Dual(-3, -10)

```
1 -Dual(1, 2) * Dual(3, 4)
```

f_1 (generic function with 1 method)

```
1 f_1(x, y) = x * y
```

f_2 (generic function with 1 method)

```
1 f_2(s1) = -s1
```

► Dual(-3, -10)

```
1 (f_2 ∘ f_1)(Dual(1, 2), Dual(3, 4))
```

Reverse Differentiation ⇌

Error message

InterruptException:

```
1 img("Blondel_Rouvet_Figure_8_3", :height => 250)
```

👁 Reading hidden code

Two different takes on the multivariate chain rule ⇌

The chain rule gives us

$$\frac{\partial f_3}{\partial x}(f_1(x), f_2(x)) = \partial_1 f_3(s_1, s_2) \cdot \frac{\partial s_1}{\partial x} + \partial_2 f_3(s_1, s_2) \cdot \frac{\partial s_2}{\partial x}$$

To compute this expression, we need the values of $s_1(x)$ and $s_2(x)$ as well as the derivatives $\partial s_1 / \partial x$ and $\partial s_2 / \partial x$.

Common to forward and reverse: Given s_1, s_2 , computes **local** derivatives $\partial_1 f_3(s_1, s_2)$ and $\partial_2 f_3(s_1, s_2)$, shortened $\partial_1 f_3, \partial_2 f_3$ for conciseness.

Forward ⇌

$$\begin{aligned} t_3 &= \partial_1 f_3 \cdot t_1 + \partial_2 f_3 \cdot t_2 \\ &= [\partial_1 f_3 \quad \partial_2 f_3] \cdot \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \\ &= \partial f_3 \cdot \begin{bmatrix} t_1 \\ t_2 \end{bmatrix} \end{aligned}$$

Reverse ⇌

$$\begin{aligned} [r_1 \quad r_2] &+= r_3 \cdot \partial f_3 \\ &+= r_3 \cdot [\partial_1 f_3 \quad \partial_2 f_3] \\ &+= [r_3 \cdot \partial_1 f_3 \quad r_3 \cdot \partial_2 f_3] \end{aligned}$$

Reverse* ⇌

$$\begin{aligned} \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} &+= \partial f_3^* \cdot r_3 \\ &+= \begin{bmatrix} \partial_1 f_3 \\ \partial_2 f_3 \end{bmatrix} \cdot r_3 \\ &+= \begin{bmatrix} \partial_1 f_3 \cdot r_3 \\ \partial_2 f_3 \cdot r_3 \end{bmatrix} \end{aligned}$$

When using automatic differentiation, don't forget that we must always evaluate the derivatives. For the following example we choose to evaluate it in $x = 3$

Error message

InterruptException:

```
1 qa(md"Apply the automatic differentiation to ``s_3=f_3(s_1, s_2) = s_1 + s_2``,  
  with ``s_1=f_1(x) = x`` and ``s_2=f_2(x) = x^2``,  
2 hbox(  
3   md""  
4   #### Forward  
5  
6   * ``\partial x / \partial x = 1``  
7   * ``\partial s_1 / \partial x = 1 \vert_{x=3} \cdot 1 = 1``  
8   * ``\partial s_2 / \partial x = 2x \vert_{x=3} \cdot 1 = 6``  
9   * ``\partial s_3 / \partial x = 1 \vert_{x=3} \cdot 1 + 1 \vert_{x=3} \cdot 6 = 7``  
10  "",  
11   Div(html" ", style = Dict("flex-grow" => "1")),  
12  md""  
13  #### Reverse  
14  
15  * Initialize ``\partial s_3 / \partial s_1 = \partial s_3 / \partial s_2 = \partial  
  s_3 / \partial x = 0``  
16  * First part: ``\partial s_3/\partial s_1 \mathrel{\raise{0.19ex}{\scriptstyle+}} =  
  1``  
17  - ``\partial s_3 / \partial x \mathrel{\raise{0.19ex}{\scriptstyle+}} = 1 \cdot 1  
  \vert_{x=3}``  
18  * Second part: ``\partial s_3/\partial s_2 \mathrel{\raise{0.19ex}{\scriptstyle+}}  
  = 1``  
19  - ``\partial s_3 / \partial x \mathrel{\raise{0.19ex}{\scriptstyle+}} = 1 \cdot  
  2x \vert_{x=3}``  
20  * The result is ``\partial s_3 / \partial x = 7``.  
21  ""))
```

 Reading hidden code

Forward tangents

Error message

InterruptException:

```
1 img("Blondel_Rouvet_Figure_8_7", :height => 250)
```

Reverse tangents ⇄

Error message

InterruptException:

```
1 img("Blondel_Rouvet_Figure_8_8", :height => 200)
```

👁 Reading hidden code

Error message

InterruptException:

```
1 qa(md"Why is ``\partial\text{dup}^*`` a sum ?", md"The Jacobian is  
  ``\partial\text{dup} = \begin{bmatrix}  
2 1\\1\\1\end{bmatrix}``. In reverse mode, we multiply by the adjoint (why ? See next  
  lecture!) of the Jacobian (here the transpose) ``\partial\text{dup}^* =  
  \begin{bmatrix}  
3 1 & 1 & 1\end{bmatrix}``. Left-multiplying a vector with a row vector of ones  
  results in its sum.")
```

👁 Reading hidden code

Computation graph ⇄

We build the graph of

$$f(x_1, x_2) = x_2 e^{x_1} \sqrt{x_1 + x_2 e^{x_2}}$$

at $x_1 = 1, x_2 = 2$. The slider first reveals the forward evaluation of each node v_i , then accumulates the adjoints $\bar{v}_i = \partial f / \partial v_i$ backwards, one node at a time.

Error message

InterruptedException:

```
1 img("Blondel_Rouvet_Figure_4_3", :height => 250)
```

 Reading hidden code

Error message

InterruptedException:

```
1 qa(md"Can this directed graph have cycles ?", md"No, it is a Directed Acyclic Graph. (DAG)")
```

 Reading hidden code

Error message

InterruptedException:

```
1 qa(md"What happens if ``f_4`` is handled before ``f_5`` in the backward pass ?",  
2 md""  
3 ``\partial f / \partial s_4`` is the sum of its contribution from ``f_5`` and  
4 ``f_7``. We must wait for ``f_5`` to be handled before we turn to ``f_4``.  
5 "")
```

 Reading hidden code

Error message

InterruptedException:

```
1 qa(md"How to prevent this from happening ?", md"We should first compute a  
[*topological ordering*](https://en.wikipedia.org/wiki/Topological_sorting) and  
then follow this order.")
```

 Reading hidden code

Reverse mode in action

Error message

InterruptedException:

```
1 @bind graph_step StepSlider(eachindex(graph_frames))
```

 Reading hidden code

Error message

InterruptedException:

```
1 HTML(CGE.render_svg(graph_example, graph_frames[graph_step]))
```

 Reading hidden code

Error message

InterruptedException:

```
1 begin
2   # One pullback per operation: given the adjoint of the output node,
3   # accumulate the adjoint of each input node.
4   function CGE.pullback! (::typeof(+), node::GraphNode, args::GraphNode...)
5       for arg in args
6           arg.metadata.derivative += node.metadata.derivative
7       end
8   end
9   function CGE.pullback! (::typeof(*), node::GraphNode, x::GraphNode, y::GraphNode)
10      x.metadata.derivative += node.metadata.derivative * y.value
11      y.metadata.derivative += node.metadata.derivative * x.value
12  end
13  function CGE.pullback! (::typeof(exp), node::GraphNode, x::GraphNode)
14      x.metadata.derivative += node.metadata.derivative * node.value
15  end
16  function CGE.pullback! (::typeof(sqrt), node::GraphNode, x::GraphNode)
17      x.metadata.derivative += node.metadata.derivative / (2 * node.value)
18  end
19 end;
```

 Reading hidden code

Comparison ⇄

- Forward mode of $f(\mathbf{x})$ with dual numbers `Dual.(x, v)` computes Jacobian-Vector Product (JVP) $J_f(\mathbf{x}) \cdot \mathbf{v}$
- Reverse mode of $f(\mathbf{x})$ computes Vector-Jacobian Product (VJP) $\mathbf{v}^\top J_f(\mathbf{x})$ or in other words $J_v(\mathbf{x})^\top \mathbf{v}$

Error message

`InterruptException:`

```
1 qa(md"How can we compute the full Jacobian ?", md"By computing a JVP (resp. VJP)
  with a one-hot vector, we get a column (resp. row) of the jacobian.")
```

👁 Reading hidden code

Error message

`InterruptException:`

```
1 qa(md"When is each mode faster than the other one to compute the full Jacobian ?",
  md"If ``f: \mathbb{R}^n \to \mathbb{R}^m``, then computing the full Jacobian
  requires ``n`` JVP or ``m`` VJP. So
2 * if ``n \gg m``, then reverse mode is faster;
3 * if ``m \gg n``, then forward mode is faster;
4 * if ``m \approx n``, then it's a close call.")
```

👁 Reading hidden code

Error message

`InterruptException:`

```
1 qa(md"When is the speed of numerical differentiation comparable to autodiff ?",
2 md""
3
```

```
4 With numerical differentiation, we compute a JVP with ``\partial f / \partial x_i
\approx (f(x_1, \ldots, x_{i-1}, x_i + \epsilon, x_{i+1}, x_n) - f(x_1, \ldots,
x_n)) / \epsilon``.
For this JVP, we need to evaluate ``f`` twice. On the other hand, forward mode
evaluates ``f`` once but with dual numbers as inputs so this evaluation is probably
around twice as expensive as evaluating ``f`` with `Float64` numbers. So the cost
of a JVP should be roughly the same for numerical differentiation and forward
5 differentiation.
6
7 Numerical differentiation may however need to increase its number of evaluations in
order to improve its accuracy while forward is accurate (up to floating point
rounding errors).
```

 Reading hidden code

Memory usage of forward mode ⇄

Error message

InterruptedException:

```
1 img("Blondel_Rouvet_Figure_8_2", :height => 250)
```

 Reading hidden code

Memory usage of reverse mode ⇄

Error message

InterruptedException:

```
1 img("Blondel_Rouvet_Figure_8_4", :height => 300)
```

 Reading hidden code

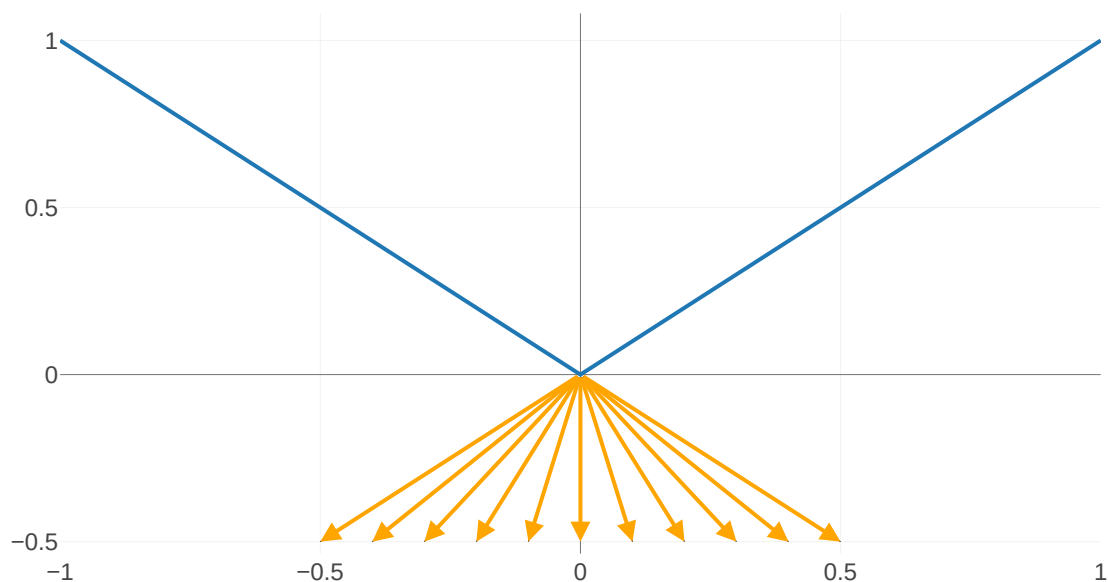
Discontinuity ↩

Error message

InterruptionException:

- 1 `qa(md"Is the function  $|x|$  is differentiable at  $x = 0$  ?.", md"No, if we approach from the left (that is,  $x < 0$ , the function is  $-x$ ), then the derivative is  $-1$ ).`
- 2 `If we approach from the right (that is,  $x > 0$ , the function is  $x$ ), then the derivative is  $1$ ).`
- 3 `There is no valid gradient!")`

👁 Reading hidden code



Error message

InterruptedException:

- 1 `qa(md"What about returning a convex combination of the derivative from the left and right ?", md"Any number between ``-1`` and ``1`` is a valid subgradient!")`
- 2 Whereas the gradient is the normal to the **unique** tangent, the subgradient is an element of the **tangent cone**, depicted below. For convex functions, the notion of subgradient appropriately generalizes the notion of gradient for nonsmooth functions.
- 3
- 4 Note that the notion of subgradient is not defined for nonconvex functions. So we may say that we compute the local subgradient of some local nonsmooth `f_i` but we cannot deduce from it that the resulting vector is a subgradient of `f` if `f` is nonconvex.")

 Reading hidden code

Forward mode ⇄

abs (generic function with 1 method)

```
1 abs(x) = ifelse(x < 0, -x, x)
```

abs_bis (generic function with 1 method)

```
1 abs_bis(x) = ifelse(x > 0, x, -x)
```

```
1 Base.isless(x::Dual, y::Real) = isless(x.value, y)
```

```
1 Base.isless(x::Real, y::Dual) = isless(x, y.value)
```

► Dual(0, 1)

```
1 abs(Dual(0, 1))
```

► Dual(0, -1)

```
1 abs_bis(Dual(0, 1))
```

Neural network ⇌

Two equivalent approaches, b_k is a **column** vector, S_i, X, W_i, Y are matrices.

Right-to-left ⇌

$$\begin{aligned} S_0 &= X \\ S_{2k-1} &= W_k S_{2k-2} + b_k \mathbf{1}^\top \\ S_{2k} &= \sigma(S_{2k-1}) \\ S_{2H+1} &= W_{k+1} S_{2H} \\ S_{2H+2} &= \ell(S_{2H+1}; Y) \end{aligned}$$

Left-to-right ⇌

$$\begin{aligned} S_0 &= X \\ S_{2k-1} &= S_{2k-2} W_k + \mathbf{1} b_k^\top \\ S_{2k} &= \sigma(S_{2k-1}) \\ S_{2H+1} &= S_{2H} W_{k+1} \\ S_{2H+2} &= \ell(S_{2H+1}; Y) \end{aligned}$$

Evaluation ⇌

Error message

InterruptException:

```
1 img("Blondel_Rouvet_Figure_8_5", :height => 150)
```

👁 Reading hidden code

Matrix multiplication (Vectorized way) ⇌

Useful: $\text{vec}(AXB) = (B^\top \otimes A)\text{vec}(X)$

$$\begin{aligned} F(X) &= AX \\ G(\text{vec}(X)) &\triangleq \text{vec}(F(X)) = (I \otimes A)\text{vec}(X) \\ J_G &= (I \otimes A) \\ J_G^\top \text{vec}(R) &= (I \otimes A^\top)\text{vec}(R) \\ \partial F^*[R] &= \text{mat}(J_G^\top \text{vec}(R)) = A^\top R \end{aligned}$$

Error message

InterruptException:

```
1 qa(md"How should we store the Jacobian in the forward pass to save it for the  
backward pass ?",  
2 md"The matrix ``I \otimes A`` is block diagonal with the same block repeated so the  
structure is crucial to exploit. Storing ``A`` is enough.")
```

 Reading hidden code

Matrix multiplication (Scalar product way) $\Leftrightarrow$

The adjoint of a linear map A for a given scalar product $\langle \cdot, \cdot \rangle$ is the linear map A^* such that

$$\forall x, y, \quad \langle A(x), y \rangle = \langle x, A^*(y) \rangle.$$

For the scalar product

$$\langle X, Y \rangle = \sum_{i,j} X_{ij} Y_{ij} = \langle \text{vec}(X), \text{vec}(Y) \rangle = \text{tr}(XY^\top), \quad A^* = A^\top$$

Now, given a forward tangent T and a reverse tangent R

$$\langle AT, R \rangle = \langle T, A^\top R \rangle$$

so the backward pass computes $A^\top R$.

Error message

InterruptException:

```
1 qa(md"How to prove that ``A^* = A^\top`` ?", md""  
2 We have ``\langle X, Y \rangle = \text{tr}(XY^\top)`` ([why ?]  
(https://en.wikipedia.org/wiki/Trace\_\(linear\_algebra\)#Trace\_of\_a\_product)) so,  
using the [cyclic property of the trace]  
(https://en.wikipedia.org/wiki/Trace\_\(linear\_algebra\)#Cyclic\_property)  
3 ``math  
4 \langle AX, Y \rangle = \text{tr}(AXY^\top) = \text{tr}(XY^\top A) = \text{tr}  
(X(A^\top Y)^\top) = \langle X, A^\top Y \rangle.  
5 ``  
6 """)
```

 Reading hidden code

Broadcasting (Vectorized way) ⇔

Consider applying a scalar function f (e.g. `tanh`) to each entry of a matrix X .

$$\begin{aligned}(F(X))_{ij} &= f(X_{ij}) = f.(X) \\ G(\text{vec}(X)) &\triangleq \text{vec}(F(X)) = \text{vec}(f.(X)) \\ J_G &= \text{Diag}(\text{vec}(f'.(X))) \\ J_G \text{vec}(T) &= \text{Diag}(\text{vec}(f'.(X))) \text{vec}(T) \\ \partial F[T] &= \text{mat}(J_G \text{vec}(T)) = f'.(X) \odot T \\ J_G^\top \text{vec}(R) &= \text{Diag}(\text{vec}(f'.(X))) \text{vec}(R) \\ \partial F^*[R] &= \text{mat}(J_G^\top \text{vec}(R)) = f'.(X) \odot R\end{aligned}$$

Broadcasting (Scalar product way) ⇔

$$\langle f'.(X) \odot T, R \rangle = \langle T, f'.(X) \odot R \rangle.$$

Error message

InterruptException:

```
1 qa(md"Let ``A(X) = B \odot X``, what is the adjoint ``A^*`` ?", md""
2 ``\math
3 \begin{align}
4   \langle B \odot X, Y \rangle
5   &=
6   \sum_{ij} ((B \odot X) \odot Y)_{ij} \\
7   &=
8   \sum_{ij} (X \odot (B \odot Y))_{ij} \\
9   &=
10  \langle X, B \odot Y \rangle.
11 \end{align}
12 ``
13 So ``A`` is self-adjoint, i.e., ``A^* = A``.
14 """)
```

👁 Reading hidden code

Error message

InterruptedException:

```
1 qa(md"What should be saved for the backward pass ?", md"The Jacobian is diagonal, we just need to compute and store the values on the diagonal: ``f.(X)`` or we can also just store ``X`` and compute ``f.(X)`` during the backward pass.")
```

 Reading hidden code

Putting everything together

Error message

InterruptedException:

```
1 img("Blondel_Rouvet_Figure_8_6", :height => 350)
```

 Reading hidden code

Product of Jacobians

Suppose that we need to differentiate a composition of functions: $(f_n \circ f_{n-1} \circ \dots \circ f_2 \circ f_1)(w)$. For each function, we can compute a jacobian given the value of its input. So, during a forward pass, we can compute all jacobians. We now just need to take the product of these jacobians:

$$J_n J_{n-1} \dots J_2 J_1$$

While the product of matrices is associative, its computational complexity depends on the order of the multiplications! Let $d_i \times d_{i-1}$ be the dimension of J_i .

Error message

InterruptedException:

```
1 qa(md"What is the complexity of forward mode", md"")
```

```

2 If the product is computed from right to left:
3 ```math
4 \begin{align}
5 J_{1,2} &= J_2 J_1 \& \Omega(d_{2d_1d_0})\\
6 J_{1,3} &= J_3 J_{1,2} \& \Omega(d_{3d_2d_0})\\
7 J_{1,4} &= J_4 J_{1,3} \& \Omega(d_{4d_3d_0})\\
8 \vdots & \quad \vdots\\
9 J_{1,n} &= J_n J_{1,(n-1)} \& \Omega(d_{nd_{n-1}d_0})
10 \end{align}
11 ```
12 we have a complexity of
13 ```\Omega(\sum_{i=2}^n d_{id_{i-1}d_0})``.
14 """

```

 Reading hidden code

Error message

InterruptException:

```

1 qa(md"What is the complexity of reverse mode", md"""
2 The adjoint trick gives
3 ```math
4 \langle J_n J_{n-1} \cdots J_2 J_1 \partial w, \partial y \rangle
5 =
6 \langle \partial w, J_1^{\top} J_2^{\top} \cdots J_{n-1}^{\top} J_n^{\top} \partial y \rangle.
7 ```
8 So reverse differentiation corresponds to multiplying the adjoint from right to left
9 or equivalently the original matrices from left to right.
10 This means computing the product in the following order:
11 ```math
12 J_{(n-1),n} = J_n J_{n-1} \& \Omega(d_{nd_{n-1}d_{n-2}})\\
13 J_{(n-2),n} = J_{(n-1),n} J_{n-2} \& \Omega(d_{nd_{n-2}d_{n-3}})\\
14 J_{(n-3),n} = J_{(n-2),n} J_{n-3} \& \Omega(d_{nd_{n-3}d_{n-4}})\\
15 \vdots \quad \vdots\\
16 J_{1,n} = J_{2,n} J_1 \& \Omega(d_{nd_1d_0})
17 \end{align}
18 ```
19 We have a complexity of
20 ```math
21 \Omega(\sum_{i=1}^{n-1} d_{nd_{i-1}}).
22 ```
23 """

```

 Reading hidden code

Error message

InterruptException:

```
1 qa(md"What about the complexity of meeting in the middle between ``k`` and
  ``k+1``?",
2   md""
3   We can also write
4   ```math
5   \langle J_n J_{n-1} \cdots J_2 J_1 \partial w, \partial y \rangle
6   =
7   \langle J_k J_{k-1} \cdots J_2 J_1 \partial w, J_{k+1}^{\text{top}} J_{k+2}^{\text{top}} \cdots J_{n-1}^{\text{top}} J_n^{\text{top}} \partial y \rangle.
8   ```
9   This corresponds to multiplying starting from some ``d_k`` where ``1 < k < n``.
10  We would then first compute the left side:
11  ```math
12  \begin{align}
13    J_{k+1,k+2} &= J_{k+2} J_{k+1} \ \& \ \Omega(d_{k+2} d_{k+1} d_k) \\
14    J_{k+1,k+3} &= J_{k+3} J_{k+1,k+2} \ \& \ \Omega(d_{k+3} d_{k+2} d_k) \\
15    \vdots & \quad \vdots \\
16    J_{k+1,n} &= J_n J_{k+1,n-1} \ \& \ \Omega(d_n d_{n-1} d_k)
17  \end{align}
18  ```
19  then the right side:
20  ```math
21  \begin{align}
22    J_{k-1,k} &= J_k J_{k-1} \ \& \ \Omega(d_k d_{k-1} d_{k-2}) \\
23    J_{k-2,k} &= J_{k-1,k} J_{k-2} \ \& \ \Omega(d_k d_{k-2} d_{k-3}) \\
24    \vdots & \quad \vdots \\
25    J_{1,k} &= J_{2,k} J_1 \ \& \ \Omega(d_k d_1 d_0)
26  \end{align}
27  ```
28  and then combine both sides:
29  ```math
30  J_{1,n} = J_{k+1,n} J_{1,k} \ \text{qqquad} \ \Omega(d_n d_k d_0)
31  ```
32  we have a complexity of
33  ```math
34  \Omega(d_n d_k d_0 + \sum_{i=1}^{k-1} d_k d_{i-1} + \sum_{i=k+2}^n d_{i-1} d_k).
35  ```
36  """)
```

👁 Reading hidden code

Error message

InterruptException:

```
1 qa(md"Which mode should be used depending on the ``d_i`` ?", md""")
2 We see that we should find the minimum ``d_k`` and start from there. If the minimum
  is attained at ``k = n``, this corresponds multiplying from left to right, this is
  reverse differentiation. If the minimum is attained at ``k = 0``, we should
  multiply from right to left, this is forward mode. Otherwise, we should start from
  the middle, this would mean mixing both forward and reverse mode.
3 """
```

 Reading hidden code

Error message

InterruptException:

```
1 qa(md"What about neural networks ?", md""")
2 In that case, ``d_0`` is equal to the number of entries in ``W_1`` added with the
  number of entries in ``W_2`` while ``d_n`` is ``1`` since the loss is scalar. We
  should therefore clearly multiply from left to right hence do reverse diff.
3 """
```

 Reading hidden code

Wine example

```
wine = dataset Wine:
  metadata => Dict{String, Any} with 4 entries
  features  => 13x178 Matrix{Float64}
  targets   => 1x178 Matrix{Int64}
  dataframe => nothing
```

```
1 wine = MLDatasets.Wine(; as_df = false)
```

normalise (generic function with 1 method)

```
1 function normalise(x)
2    $\mu$  = Statistics.mean(x, dims=2)
3    $\sigma$  = Statistics.std(x, dims=2, mean= $\mu$ )
4   return (x .-  $\mu$ ) ./  $\sigma$ 
5 end
```

```
X = 13x178 Matrix{Float32}:
 1.51434  0.245597  0.196325  1.68679  ...  0.331822  0.208643  1.39116
-0.560668 -0.498009  0.0211715 -0.345835  1.73984  0.227053  1.57871
 0.2314   -0.825667  1.10621   0.486554  -0.38826  0.0126963  1.36137
-1.1663   -2.48384  -0.267982 -0.806975  0.151234  0.151234  1.49872
 1.90852   0.018094  0.0881098  0.9283    1.41841  1.41841  -0.261969
 0.806722  0.567048  0.806722  2.48444  ... -1.12665  -1.03078  -0.391646
 1.03191   0.731565  1.21211   1.4624    -1.3408  -1.35081  -1.27072
-0.657708 -0.818411  -0.497005 -0.979113  0.547563  1.35108  1.59213
 1.22144  -0.543189  2.12996   1.02925  -0.420888 -0.228701 -0.420888
 0.251009 -0.292496  0.268263  1.18273   2.21798  1.82976  1.78663
 0.361158  0.404908  0.317409 -0.426341  ... -1.60759  -1.56384  -1.52009
 1.84272   1.11032  0.786369  1.18074  -1.48127  -1.39676  -1.42493
 1.01016   0.962526  1.39122   2.32801   0.279786  0.295664  -0.593486
```

```
1 X = Float32.(normalise(wine.features))
```

```
y =
1x178 Matrix{Float32}:
-1.0 -1.0 -1.0 -1.0 -1.0 -1.0 -1.0 ... 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0
```

```
1 y = Float32.(wine.targets .- 2)
```

Neural network

h =  16

```
W =
▶ [16×13 Matrix{Float32}:
  0.966676  0.847825  0.457134  0.210799  ...  0.418842  0.659798  0.708644  0.420
1 W = [rand(Float32, h, size(X, 1)), rand(Float32, size(y, 1), h)]
```

```
53.98862f0
1 sum((W[2] * tanh.(W[1] * X) - y).^2) / size(y, 2)
```

Forward mode ⇄

forward_pass (generic function with 1 method)

```
1 function forward_pass(W, X, y)
2     W1, W2 = W
3     y_1 = tanh.(W1 * X)
4     local_der_tanh = 1 .- y_1.^2
5     local_der_mse = 2 * (W2 * y_1 - y) / size(y, 2)
6     return local_der_tanh, local_der_mse
7 end
```

forward_diff (generic function with 1 method)

```
1 function forward_diff(W, X, y, j, k)
2     W1, W2 = W
3     T_1 = onehot(j, axes(W1, 1)) * onehot(k, axes(W1, 2))'
4     J_1, J_2 = forward_pass(W, X, y)
5     only((W2 * (J_1 .* (T_1 * X))) * J_2') # only: 1x1 matrix -> scalar
6 end
```

forward_diff (generic function with 2 methods)

```
1 function forward_diff(W, X, y)
2     [forward_diff(W, X, y, i, j) for i in axes(W[1], 1), j in axes(W[1], 2)]
3 end
```

Error message

```
Malt.TerminatedWorkerException()
```

```
1 if h < 200 # Forward Diff start being too slow for 'h > 200'
2     @time forward_diff(W, X, y)
3 end
```

Reverse mode

Error message

InterruptException:

```
1 qa(md"How to deduce the backward pass for reverse mode from the forward mode ?",
md""
2 `W2 * (J_1 .* (T_1 * X))) * J_2'` → The broadcasted `*` is an Hadamard product,
denoted  $\odot$ . So forward mode is:
3 ```math
4 W_2 (J_1 \odot (T_1 X)) J_2^{\text{top}}
5 ```
6 As this is scalar, it is trivially equal to its scalar product with  $\mathbf{1}$ . That is,
we start with a reverse tangent  $R = 1$ . We can then move everything to the right-
hand side with the adjoint (here, it is the transpose):
7 ```math
8 \begin{align}
9 \langle W_2 (J_1 \odot (\partial W_1 X)) J_2^{\text{top}}, 1 \rangle
10 &=
11 \langle J_1 \odot (\partial W_1 X), W_2^{\text{top}} J_2 \rangle \\
12 &=
13 \langle \partial W_1 X, J_1 \odot (W_2^{\text{top}} J_2) \rangle \\
14 &=
15 \langle \partial W_1, (J_1 \odot (W_2^{\text{top}} J_2)) X^{\text{top}} \rangle \\
16 \end{align}
17 ```
18 Now, for the derivative with respect to the entry  $W_{i,j}$ , we use  $\partial W_1 = e_{i,j}^{\text{top}}$  and
19 ```math
20 \langle e_{i,j}^{\text{top}}, (J_1 \odot (W_2^{\text{top}} J_2)) X^{\text{top}} \rangle
21 =
22 ((J_1 \odot (W_2^{\text{top}} J_2)) X^{\text{top}})_{ij}
23 ```
24 So the gradient with respect to  $W_1$  is exactly the matrix  $(J_1 \odot (W_2^{\text{top}} J_2)) X^{\text{top}}$  !
25 """)
```

 Reading hidden code

reverse_diff (generic function with 1 method)

```
1 function reverse_diff(W, X, y)
2     J_1, J_2 = forward_pass(W, X, y)
3     (J_1 .* (W[2]' * J_2)) * X'
4 end
```

```
16x13 Matrix{Float32}:
0.0645606 -0.0911188 0.169858 ... 0.0557593 0.070503 0.106306
1.18888 0.164423 0.0783756 -0.0804011 0.77867 0.976297
0.235812 0.167953 0.0635253 -0.187064 0.0374819 0.271761
0.0475736 -0.291443 0.212598 0.488191 0.524017 0.179939
0.0775779 0.231821 0.257064 0.0670342 0.0298729 0.00750218
0.0628102 -0.171433 0.374583 ... 0.897635 1.13955 0.417715
0.358311 -0.257757 0.362325 0.882258 1.23481 0.991299
⋮
0.161662 -0.441274 -0.0804704 ... 0.514898 0.600549 0.414796
0.665313 -0.368924 0.290678 0.481751 0.980018 0.975922
0.957592 0.238773 -0.176458 -0.0736579 0.627156 1.11313
0.121801 -0.151097 -0.00257029 -0.144563 0.388381 0.283363
0.600845 0.225726 0.485821 -0.0552279 -0.314507 0.616632
0.399888 -0.444734 0.150511 ... 0.488588 0.623945 0.401869
```

```
1 @time reverse_diff(W, X, y)
```



```
0.000803 seconds (25 allocations: 60.039 KiB)
```



GPU acceleration ⚡

```
1 if CUDA.functional()
2     X_gpu = CUDA.CuArray(X)
3     y_gpu = CUDA.CuArray(y)
4     W_gpu = CUDA.CuArray(W)
5     @time reverse_diff(W_gpu, X_gpu, y_gpu)
6 end
```

Error message

InterruptException:

```
1 qa(md"Why is the GPU version slower than the CPU version ?",
2 md"")
3 The fixed cost of launching the GPU kernels is larger than the gain obtained by the
4 GPU acceleration. It is only worth it for larger `h`.
5 `h` = $(h_slider).
```

5 """)

👁 Reading hidden code

Second-order

Consider a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, we want to compute the Hessian $\nabla^2 f(x)$, defined by

$$(\nabla^2 f(x))_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Application: Given the optimization problem:

$$\begin{aligned} \min_x f(x) \\ g_i(x) = 0 \quad \forall i \in \{1, \dots, m\} \end{aligned}$$

The Hessian of the Lagrangian $\mathcal{L}(x, \lambda) = f(x) - \lambda_1 g_1(x) - \dots - \lambda_m g_m(x)$ is obtained as

$$\nabla_x^2 \mathcal{L}(x, \lambda) = \nabla^2 f(x) - \sum_{i=1}^m \lambda_i \nabla^2 g_i(x)$$

Second-order AD

Error message

InterruptException:

```
1 ga(md"How can the Hessian of ``f`` be computed given an AD for Jacobian and  
  gradient.", md""  
2 Define the function ``g(x) = \nabla f(x)`` , the Hessian of ``f`` is then the  
  Jacobian of ``g``:  
3 ``\nabla^2 f(x) = J_g(x)``. See the solutions of `LabAD`!  
4 """)
```

 Reading hidden code

Error message

InterruptedException:

```
1 qa(md"Does the AD need to be the same for the gradient and the Jacobian ?",
2 md""
3 No. Given an implementation of reverse mode and forward mode, this gives 4
4 possibilities depending on which mode is used for the gradient and Jacobian.
5 """)
```

👁 Reading hidden code

Notation ⇌

- Let $f_k : \mathbb{R}^{d_{k-1}} \rightarrow \mathbb{R}^{d_k}$. $\partial f_k \triangleq \partial f_k(s_{k-1}) \in \mathbb{R}^{d_k \times d_{k-1}}$, $\partial^2 f_k \triangleq \partial^2 f_k(s_{k-1}) \in \mathbb{R}^{d_k \times d_{k-1} \times d_{k-1}}$ is a 3D array/tensor.
- Given $v \in \mathbb{R}^{d_{k-1}}$, by the product $(\partial^2 f_k \cdot v) \in \mathbb{R}^{d_k \times d_{k-1}}$, we denote the contraction of the 3rd (or 2nd since the tensor is symmetric over its last 2 dimensions) dimension:

$$(\partial^2 f_k \cdot v)_{ij} = \sum_{l=1}^{d_{k-1}} (\partial^2 f_k)_{ijl} \cdot v_l$$

- Given $u \in \mathbb{R}^{d_k}$, by the product $(u \cdot \partial^2 f_k) \in \mathbb{R}^{d_{k-1} \times d_{k-1}}$, we denote the contraction of the 1st dimension.

$$(u \cdot \partial^2 f_k)_{ij} = \sum_{l=1}^{d_k} u_l \cdot (\partial^2 f_k)_{lij}$$

- Both $\partial^2 f_k \cdot v$ and $u \cdot \partial^2 f_k$ are matrices so then we're back to matrix notations.

Chain rule ⇌

$$\begin{aligned} \frac{\partial^2 (f_2 \circ f_1)}{\partial x_i \partial x_j} &= \frac{\partial}{\partial x_j} \left(\frac{\partial (f_2 \circ f_1)}{\partial x_i} \right) \\ &= \frac{\partial}{\partial x_j} \left(\partial f_2 \cdot \frac{\partial f_1}{\partial x_i} \right) \\ &= \left(\partial^2 f_2 \cdot \frac{\partial f_1}{\partial x_j} \right) \cdot \frac{\partial f_1}{\partial x_i} + \partial f_2 \cdot \frac{\partial^2 f_1}{\partial x_i \partial x_j} \end{aligned}$$

In terms of the matrices $J_k = \partial f_k$ and $H_{kj} = \frac{\partial}{\partial x_j} J_k = \partial^2 f_k \cdot \frac{\partial s_{k-1}}{\partial x_j}$, it becomes

$$\frac{\partial^2(f_2 \circ f_1)}{\partial x_i \partial x_j} = H_{2j} \cdot \frac{\partial f_1}{\partial x_i} + J_2 \cdot \frac{\partial^2 f_1}{\partial x_i \partial x_j}$$

Forward on forward ⇔

Given $\text{Dual}(\mathbf{s}_1, t_1)$ with $\mathbf{s}_1 = \text{Dual}(f_1(x), \frac{\partial f_1}{\partial x_j})$ and $t_1 = \text{Dual}(\frac{\partial f_1}{\partial x_i}, \frac{\partial^2 f_1}{\partial x_i \partial x_j})$

1. Compute $\mathbf{s}_2 = f_2(\mathbf{s}_1) = (f_2(f_1(x)), J_2 \cdot \frac{\partial f_1}{\partial x_j}) = ((f_2 \circ f_1)(x), \partial(f_2 \circ f_1)/\partial x_j)$
2. Compute $J_{f_2}(\mathbf{s}_1)$ which gives $\text{Dual}(J_2, H_{2j})$
3. Compute

$$\begin{aligned} J_{f_2}(\mathbf{s}_1) \cdot t_1 &= \text{Dual}(J_2, H_{2j}) \cdot \text{Dual}(\frac{\partial f_1}{\partial x_i}, \frac{\partial^2 f_1}{\partial x_i \partial x_j}) \\ &= \text{Dual}(J_2 \cdot \frac{\partial f_1}{\partial x_i}, J_2 \cdot \frac{\partial^2 f_1}{\partial x_i \partial x_j} + H_{2j} \cdot \frac{\partial f_1}{\partial x_i}) \\ &= \text{Dual}(\frac{\partial(f_2 \circ f_1)}{\partial x_i}, \frac{\partial^2(f_2 \circ f_1)}{\partial x_i \partial x_j}) \end{aligned}$$

Error message

InterruptException:

```
1 qa(md"What is the closed form expression for ``t_k`` in terms of the matrices  
  ``J_k`` and ``H_{kj}`` ?",  
2 md""  
3 We can prove by induction that the second part of the dual number ``t_k`` is:  
4 ``math  
5 \frac{\partial^2 f_k}{\partial x_i \partial x_j} = J_k \cdot J_2 \cdot H_{1j} \cdot e_i + J_k  
  \cdot J_3 \cdot H_{2j} \cdot J_1 \cdot e_i + H_{kj} \cdot J_{k-1} \cdot J_1 \cdot e_i  
6 ``  
7 """)
```

 Reading hidden code

Forward on reverse ⇔

Forward pass: Given $\mathbf{s}_1 = \text{Dual}(f_1(x), \frac{\partial f_1}{\partial x_j})$

1. Compute $\mathbf{s}_2 = \mathbf{f}_2(\mathbf{s}_1) \rightarrow$ same as forward on forward
2. Compute $\mathbf{J}_{\mathbf{f}_2}(\mathbf{s}_1) \rightarrow$ same as forward on forward

Reverse pass: Given $\mathbf{r}_2 = \text{Dual}((\mathbf{r}_2)_1, (\mathbf{r}_2)_2)$, compute

$$\begin{aligned} \mathbf{r}_2 \cdot \mathbf{J}_2 &= \text{Dual}((\mathbf{r}_2)_1, (\mathbf{r}_2)_2) \cdot \text{Dual}(\mathbf{J}_2, \mathbf{H}_{2j}) \cdot \\ &= \text{Dual}((\mathbf{r}_2)_1 \cdot \mathbf{J}_2, (\mathbf{r}_2)_2 \cdot \mathbf{J}_2 + (\mathbf{r}_2)_1 \cdot \mathbf{H}_{2j}) \end{aligned}$$

Error message

InterruptException:

```

1  qa(md"Which value of ``r_k`` is solution for this recurrence equation ?",
2  md""
3  We find ``r_k = \text{Dual}(\frac{\partial f}{\partial s_k}, \frac{\partial^2 f}{\partial^2 s_k \partial x_j})`` as solution:
4  ```math
5  \begin{align}
6  (r_1)_2 &= (r_2)_2 \cdot J_2 + (r_2)_1 \cdot H_{2j} \\
7  &= \frac{\partial^2 f}{\partial s_2 \partial x_j} \cdot J_2 + \\
8  &\frac{\partial f}{\partial s_2} \cdot H_{2j} \\
9  &= \frac{\partial^2 f}{\partial s_2 \partial x_j} \frac{\partial s_2}{\partial s_1} \\
10 &+ \\
11 &\frac{\partial f}{\partial s_2} \frac{\partial J_2}{\partial x_j} \\
12 &= \frac{\partial^2 f}{\partial s_2 \partial x_j} \frac{\partial s_2}{\partial s_1} \frac{\partial J_2}{\partial x_j} \\
13 &+ \frac{\partial f}{\partial x_j} \left( \frac{\partial f}{\partial s_2} \frac{\partial J_2}{\partial x_j} \right) \\
14 &= \frac{\partial^2 f}{\partial s_1 \partial x_j} \\
15 \end{align}
16 ```
17 """)

```

 Reading hidden code

Error message

InterruptException:

```

1 qa(md"What is the closed form expression for ``r_k`` in terms of the matrices
  ``J_k`` and ``H_{kj}`` ?",
2 md""
3 We can prove by induction that the second part of the dual number ``r_k`` is:
4 ``math
5 \frac{\partial^2 f}{\partial s_k \partial x_j}^{\text{top}} = (J_K \cdots J_{k+1} H_{kj} +
  J_K \cdots J_{k+2} H_{(k+1)j} J_k + H_{Kj} J_{K-1} \cdots J_k)^{\text{top}}
6 ``
7 """)

```

👁 Reading hidden code

Reverse on forward ⇄

Forward pass: Given $s_1 = \text{Dual}(f_1(x), \frac{\partial f_1}{\partial x_i})$

1. Forward mode computes

$$s_2 = f_2(s_1) = (f_2(f_1(x)), J_2 \cdot \frac{\partial f_1}{\partial x_i}) = ((f_2 \circ f_1)(x), \partial(f_2 \circ f_1)/\partial x_i)$$

2. The reverse mode computes the local Jacobian of this operation : $\partial s_2 / \partial s_1$. The local Jacobian of $(s_1)_1 \mapsto f_2((s_1)_1)$ is J_2 . The local Jacobian of $s_1 \mapsto \partial f_2((s_1)_1)(s_1)_2$ is $(\partial^2 f_2((s_1)_1) \cdot (s_1)_2, \partial f_2((s_1)_1)) = (\partial^2 f_2(f_1(x)) \cdot \frac{\partial f_1}{\partial x_i}, \partial f_2(f_1(x))) = (H_{2i}, J_2)$

Reverse pass:

$$\begin{aligned} (r_1)_1 &= (r_2)_1 \cdot J_2 + (r_2)_2 \cdot H_{2i} \\ (r_1)_2 &= (r_2)_2 \cdot J_2 \end{aligned}$$

Error message

InterruptException:

```

1 qa(md"Which value of ``r_k`` is solution for this recurrence equation ?",
2 md""
3 We find ``r_k = \text{Dual}(\frac{\partial f}{\partial s_k}, \frac{\partial^2 f}{\partial s_k \partial x_{\textcolor{red}{j}}})`` as solution.
4 """)

```

👁 Reading hidden code

Reverse on reverse ⇄

Forward pass (2nd):

1. Forward pass computes $\mathbf{s}_2 = \mathbf{f}_2(\mathbf{s}_1) \rightarrow$ Jacobian $\partial \mathbf{s}_2 / \partial \mathbf{s}_1 = \mathbf{J}_2$
2. Local Jacobian $\mathbf{J}_2 = \partial \mathbf{f}_2(\mathbf{s}_1) \rightarrow$ The Jacobian is the 3D array $\partial \mathbf{J}_2 / \partial \mathbf{s}_1 = \partial^2 \mathbf{f}_2$
3. Backward pass computes $\mathbf{r}_1 = \mathbf{r}_2 \cdot \partial \mathbf{f}_2(\mathbf{s}_1) \rightarrow$ Jacobian of $(\mathbf{s}_1, \mathbf{r}_2) \mapsto \mathbf{r}_2 \cdot \partial \mathbf{f}_2(\mathbf{s}_1)$ is $(\mathbf{r}_2 \cdot \partial^2 \mathbf{f}_2(\mathbf{s}_1), \partial \mathbf{f}_2(\mathbf{s}_1)) = (\mathbf{r}_2 \cdot \partial^2 \mathbf{f}_2, \mathbf{J}_2)$. Note that here $\mathbf{r}_2 \in \mathbb{R}^{d_k}$ is multiplying the first dimension of the tensor $\partial^2 \mathbf{f}_2(\mathbf{s}_1) \in \mathbb{R}^{d_k \times d_{k-1} \times d_{k-1}}$ so the result is a symmetric matrix of dimension $\mathbb{R}^{d_{k-1} \times d_{k-1}}$

Reverse pass (2nd): The result is $\mathbf{r}_0$, let $\dot{\mathbf{r}}_k$ be the second-order reverse tangent for $\mathbf{r}_k$ and $\dot{\mathbf{s}}_k$ be the second-order reverse tangent of $\mathbf{s}_k$. We have

$$\begin{aligned}\dot{\mathbf{r}}_2 &= \mathbf{J}_2 \cdot \dot{\mathbf{r}}_1 \\ \dot{\mathbf{s}}_1 &= (\mathbf{r}_2 \cdot \partial^2 \mathbf{f}_2(\mathbf{s}_1)) \cdot \dot{\mathbf{r}}_1 + \dot{\mathbf{s}}_2 \cdot \mathbf{J}_2\end{aligned}$$

Error message

InterruptException:

```
1 qa(md"Which value of ``\dot{s}_k, \dot{r}_k`` is solution for this recurrence
  equation ?",
2 md""
3 Starting with ``\dot{s}_0 = e_i``, we have
4 ``math
5 \begin{align}
6 r_k &= J_K \cdots J_{k+1} \\
7 \dot{r}_k &= J_k \cdots J_1 e_i \\
8 (r_k \cdot \partial^2 f_k) \cdot \dot{r}_{k-1} \\
9 &= \\
10 r_k \cdot (\partial^2 f_k \cdot \dot{r}_{k-1}) \\
11 &= \\
12 r_k \cdot H_{ki} \\
13 \dot{s}_k &= \sum_{k=1}^K r_k H_{ki} J_{k-1} \cdots J_1 \\
14 \end{align}
15 ``
16 So we find ``\dot{s}_k = \frac{\partial^2 f}{\partial s_k \partial x_i}, \dot{r}_k
  = \frac{\partial f}{\partial s_k}`` as solution.
17 """)
```

 Reading hidden code

Error message

InterruptedException:

```
1 qa(md"What is the difference with reverse on forward and forward on reverse ?",
2 md"""
3 Reverse on reverse computes the product in the order
4 ```math
5 ((J_K \cdots J_{k-1}) \cdot \partial^2 f_k) \cdot (J_k \cdots J_1 e_i)
6 ```
7 while reverse on forward and forward on reverse compute it in the order
8 ```math
9 (J_K \cdots J_{k-1}) \cdot (\partial^2 f_k \cdot (J_k \cdots J_1 e_i))
10 ```
11 """)
```

 Reading hidden code

Acknowledgements and further readings

- Dual is inspired from [ForwardDiff](#)
- Node is inspired from [micrograd](#)
- [Here](#) is a good intro to AD
- Figures are from the [The Elements of Differentiable Programming book](#)

The End

Utils

```
1 using PlutoUI, PlutoUI.ExperimentalLayout, HypertextLiteral, PlutoTeachingTools,  
   DataFrames, MLDatasets, Statistics, CUDA, OneHotArrays
```

```
1 import PlotlyLight
```

```
1 import ComputationGraphExplorer as CGE
```

Error message

InterruptException:

```
1 begin  
2     # The graph carries the primal value; the adjoint is user metadata.  
3     mutable struct Adjoint  
4         derivative::Float64  
5     end  
6     CGE.metadata{::Type{Adjoint}, ::Float64} = Adjoint{0.0}  
7     CGE.metadata_rows(data::Adjoint) = ["adjoint" => data.derivative]  
8     CGE.seed_metadata!(data::Adjoint, is_output::Bool) =  
9         data.derivative = is_output ? 1.0 : 0.0  
10    const GraphNode = CGE.Node{Float64,Adjoint}  
11 end;
```

 Reading hidden code

Error message

InterruptException:

```
1 graph_example = let  
2     x1 = GraphNode(1.0)  
3     x2 = GraphNode(2.0)  
4     v1 = exp(x1)  
5     v2 = exp(x2)  
6     v3 = x2 * v2  
7     v4 = x1 + v3
```

```

8   v5 = sqrt(v4)
9   v6 = x2 * v1
10  f = v6 * v5
11  names = IdDict(
12      x1 => "x1", x2 => "x2",
13      v1 => "v1", v2 => "v2", v3 => "v3",
14      v4 => "v4", v5 => "v5", v6 => "v6",
15      f => "f",
16  )
17  CGE.Graph(f; names)
18 end;

```

 Reading hidden code

Error message

InterruptedException:

```

1  graph_frames = let
2      frames = CGE.forward_frames(graph_example)
3      order = CGE.topological_order(graph_example.output)
4      for node in order
5          node.metadata.derivative = 0.0
6      end
7      graph_example.output.metadata.derivative = 1.0
8      push!(frames, CGE.capture_frame(
9          graph_example, "Reverse pass: seed  $\bar{f} = 1$ "; active = graph_example.output,
10     ))
11     for node in reverse(order)
12         isempty(node.args) && continue
13         CGE.pullback!(node)
14         push!(frames, CGE.capture_frame(
15             graph_example,
16             "Reverse pass: propagate from $(graph_example.names[node])";
17             active = node,
18         ))
19     end
20     frames
21 end;

```

 Reading hidden code

Error message

InterruptedException:

```

1 begin
2   struct Path
3     path::String
4   end
5
6   function imgpath(path::Path)
7     file = path.path
8     if !('.' in file)
9       file = file * ".png"
10    end
11    return joinpath(joinpath(@__DIR__, "images", file))
12  end
13
14  function img(path::Path, args...; kws...)
15    return PlutoUI.LocalResource(imgpath(path), args...)
16  end
17
18  struct URL
19    url::String
20  end
21
22  function save_image(url::URL, html_attributes...; name = split(url.url, '/') [end],
23    kws...)
24    path = joinpath("cache", name)
25    return PlutoTeachingTools.RobustLocalResource(url.url, path,
26      html_attributes...), path
27  end
28
29  function img(url::URL, args...; kws...)
30    r, _ = save_image(url, args...; kws...)
31    return @html("<a href=$(url.url)>$r</a>")
32  end
33
34  function img(file::String, args...; kws...)
35    if startswith(file, "http")
36      img(URL(file), args...; kws...)
37    else
38      img(Path(file), args...; kws...)
39    end
40  end
41 end

```

👁 Reading hidden code

Error message

InterruptedException:

```
1 begin
2 function qa(question, answer)
3     return @html("<details><summary>$question</summary>$answer</details>")
4 end
5 function _inline_html(m::Markdown.Paragraph)
6     return sprint(Markdown.htmlinline, m.content)
7 end
8 function qa(question::Markdown.MD, answer)
9     # 'html(question)' will create '<p>' if 'question.content[]' is
10    # 'Markdown.Paragraph'
11    # This will print the question on a new line and we don't want that:
12    h = HTML(_inline_html(question.content[]))
13    return qa(h, answer)
14 end
```

👁 Reading hidden code